

```
In [ ]: import xml.etree.ElementTree as ET

import marimo as mo
import pandas as pd
```

Data Dictionary

Predictors

```
In [ ]: def print_sections(items):
    # PDF/print exports have no JS, so tabs/accordions can't hide panes
    # interactively - render every item as an always-visible labeled
    # section instead, which also reads fine in the live app.
    return mo.vstack(
        [
            mo.vstack([mo.md(f"**{label}**"), content])
            for label, content in items.items()
        ]
    )
```

```
In [ ]: data_dir = mo.notebook_dir() / ".." / "data" / "usgs"

if not data_dir.exists():
    # Notebook was opened standalone (e.g. `uvx marimo edit --sandbox
    # <gh-pages-url>`), so there's no sibling data/ checkout. Download the
    # same files from the repo into a local cache and use that instead.
    import tempfile
    import urllib.request
    from pathlib import Path

    data_dir = Path(tempfile.gettempdir()) / "egsy-pfas-data" / "usgs"
    _RAW_BASE = "https://raw.githubusercontent.com/egsy-intell/team-project/"
    _files = [
        "seawolf/NationalPFASReconLandscapeMetadata.xml",
        "mcmahon/PFAS_Data_Dictionary.csv",
        "mcmahon/PFAS_ENV.csv",
    ]
    for _rel in _files:
        _dest = data_dir / _rel
        if _dest.exists():
            continue
        _dest.parent.mkdir(parents=True, exist_ok=True)
        urllib.request.urlretrieve(f"{_RAW_BASE}/{_rel}", _dest)

seawolf_meta_tree = ET.parse(
    data_dir / "seawolf" / "NationalPFASReconLandscapeMetadata.xml"
)

mcmahon_dict_df = pd.read_csv(
    data_dir / "mcmahon" / "PFAS_Data_Dictionary.csv", encoding="latin1"
)
```

```

mcmahon_env_df = pd.read_csv(data_dir / "mcmahon" / "PFAS_ENV.csv")

# Data quirk: All values are labeled with <<compound>>-VA
# except for "PFBS-V". This is a correction
mcmahon_env_df = mcmahon_env_df.rename(columns={"PFBS-V": "PFBS-VA"})

# McMahon spells some compounds differently than Smalling despite
# referring to the same substance; normalize to Smalling's spelling so
# shared compounds line up as a single row.
mcmahon_alias = {
    "4_2 FTS": "4:2 FTS",
    "6_2 FTS": "6:2 FTS",
    "8_2 FTS": "8:2 FTS",
    "PFOSA": "FOSA",
    "PFOS ": "PFOS",
}

# mcmahon_env_df's "-VA"/"-RMK" columns use McMahon's original spelling;
# align them here so downstream notebooks receive a dataframe that's
# already consistent with all_compound_dict_df, without needing
# mcmahon_alias itself.
mcmahon_env_df = mcmahon_env_df.rename(
    columns={
        f"{old}-{suffix}": f"{new}-{suffix}"
        for old, new in mcmahon_alias.items()
        for suffix in ("VA", "RMK")
    }
)

def _to_float(value):
    return float(value) if value is not None else None

seawolf_dict_df = pd.DataFrame(
    {
        "attribute": attr.findtext("attrlabl"),
        "definition": attr.findtext("attrdef"),
        "rdom_min": _to_float(attr.findtext("./rdom/rdommin")),
        "rdom_max": _to_float(attr.findtext("./rdom/rdommax")),
    }
    for attr in seawolf_meta_tree.findall("./eainfo/detailed/attr")
)

def filter_mcmahon(table_name):
    filter = mcmahon_dict_df["TABLE"] == table_name

    return mcmahon_dict_df.loc[filter, ["PARAMETER", "DEFINITION"]].apply(
        lambda col: col.str.strip()
    )

print_sections(
    {"Seawolf": seawolf_dict_df, "McMahon": filter_mcmahon("PFAS_ENV")}
)

```

Seawolf

	attribute	definition	rdom_min	rdom_max
0	station_nm	Project defined site names for each tapwater sample collected, 2016-2021	NaN	NaN
1	Study	Name of study	NaN	NaN
2	number_pfas_sites_proximal	Number of PFAS source facilities identified from EPA PFAS Analytics Tools within a 5km buffer; individual facility information can be found in PFAS_ECHOfacilities_5km.csv	1.000000	214.000000
3	mean_dist_to_pfas_site	Mean distance to PFAS source facilities within the 5km buffer; individual facility information can be found in PFAS_ECHOfacilities_5km.csv	105.265881	4997.394570
4	Burn_Area_5k_frac	Fraction of 5km buffer that is estimated as burned as defined by MTBS. Calculated as area burned divided by total 5km buffer area	0.000637	0.949070
...	...	...	...	...
30	Facility	Manufacturing or chemical production facilities that produce or use PFAS	NaN	NaN
31	Industry	Type of production or manufacturing occurring at the listed facility	NaN	NaN
32	ECHO_Facility_Report	Link to the ECHO facility report	NaN	NaN
33	EJSCREEN_Report	Link to the EJSCREEN report	NaN	NaN
34	EPA_Programs	Type of program listed by the EPA	NaN	NaN

35 rows × 4 columns

McMahon

	PARAMETER	DEFINITION
35		Concentrations of per- and polyfluoroalkyl substances (PFAS) in environmental samples
36	NAWQA_ID	NAWQA well identification number
37	DATE	Sample collection date

	PARAMETER	DEFINITION
38	TIME	Sampe collection time
39	<parameter name>-RMK	Remark code for indicated parameter, if needed
...	...	...
63	PFPeS	Perfluoropentane sulfonate (PFPeS), linear and branched, water, unfiltered, recoverable, nanograms per liter
64	PFPeA	Perfluoropentanoate (PFPeA), linear and branched, water, unfiltered, recoverable, nanograms per liter
65	PFTeDA	Perfluorotetradecanoate (PFTeDA), linear and branched, water, unfiltered, recoverable, nanograms per liter
66	PFTTrDA	Perfluorotridecanoate (PFTTrDA), linear and branched, water, unfiltered, recoverable, nanograms per liter
67	PFUnA	Perfluoroundecanoate (PFUnA), linear and branched, water, unfiltered, recoverable, nanograms per liter

33 rows × 2 columns

Compounds

```
In [ ]: # McMahon's PFAS_ENV table mixes non-compound fields (NAWQA_ID, DATE, TIME)
# and remark-code boilerplate rows in with the actual PFAS compounds, so
# filter down to rows whose PARAMETER is an actual compound abbreviation.
_mcmahon_non_compound = {
    "",
    "NAWQA_ID",
    "DATE",
    "TIME",
    "<parameter name>-RMK",
    "<parameter name>-VA",
    "<",
    "E",
    "n",
}

_mcmahon_compound_df = (
    filter_mcmahon("PFAS_ENV")
    .rename(columns={"PARAMETER": "compound", "DEFINITION": "definition"})
    .loc[lambd df: ~df["compound"].isin(_mcmahon_non_compound)]
    .assign(compound=lambd df: df["compound"].replace(mcmahon_alias))
    .reset_index(drop=True)
)

# Smalling's PFAS_ENV.csv has no accompanying data dictionary, so the
# compound names are documented here directly.
_smalling_compound_df = pd.DataFrame(
    [
        {
```

```
    "compound": "PFBA",
    "definition": "Perfluorobutanoate (PFBA), nanograms per liter",
  },
  {
    "compound": "PFPeA",
    "definition": (
      "Perfluoropentanoate (PFPeA), nanograms per liter"
    ),
  },
  {
    "compound": "PFHxA",
    "definition": (
      "Perfluorohexanoate (PFHxA), nanograms per liter"
    ),
  },
  {
    "compound": "PFHpA",
    "definition": (
      "Perfluoroheptanoate (PFHpA), nanograms per liter"
    ),
  },
  {
    "compound": "PFOA",
    "definition": "Perfluorooctanoate (PFOA), nanograms per liter",
  },
  {
    "compound": "PFNA",
    "definition": "Perfluorononanoate (PFNA), nanograms per liter",
  },
  {
    "compound": "PFDA",
    "definition": "Perfluorodecanoate (PFDA), nanograms per liter",
  },
  {
    "compound": "PFBS",
    "definition": (
      "Perfluorobutane sulfonate (PFBS), nanograms per liter"
    ),
  },
  {
    "compound": "PFPeS",
    "definition": (
      "Perfluoropentane sulfonate (PFPeS), nanograms per liter"
    ),
  },
  {
    "compound": "PFHxS",
    "definition": (
      "Perfluorohexane sulfonate (PFHxS), nanograms per liter"
    ),
  },
  {
    "compound": "PFHpS",
    "definition": (
      "Perfluoroheptane sulfonate (PFHpS), nanograms per liter"
    ),
  },
}
```

```

    },
    {
        "compound": "PFOS",
        "definition": (
            "Perfluorooctane sulfonate (PFOS), nanograms per liter"
        ),
    },
    {
        "compound": "PFDS",
        "definition": (
            "Perfluorodecane sulfonate (PFDS), nanograms per liter"
        ),
    },
    {
        "compound": "PFPrS",
        "definition": (
            "Perfluoropropane sulfonate (PFPrS), nanograms per liter"
        ),
    },
    {
        "compound": "6:2 FTS",
        "definition": (
            "6:2 Fluorotelomer sulfonate (6:2 FTS), nanograms per "
            "liter"
        ),
    },
    {
        "compound": "FOSA",
        "definition": (
            "Perfluorooctane sulfonamide (FOSA, also called "
            "PFOSA), nanograms per liter"
        ),
    },
    {
        "compound": "HFPO-DA;GenX",
        "definition": (
            "Hexafluoropropylene oxide dimer acid (HFPO-DA, trade "
            "name GenX), nanograms per liter"
        ),
    },
]
)

all_compound_dict_df = (
    pd.merge(
        _mcmahon_compound_df,
        _smalling_compound_df,
        on="compound",
        how="outer",
        suffixes=("_mcmahon", "_smalling"),
        indicator=True,
    )
    .assign(
        definition=lambda df: df["definition_mcmahon"].fillna(
            df["definition_smalling"]
        ),
    ),

```

```

        mcmahon=lambda df: df["_merge"].isin(["left_only", "both"]),
        smalling=lambda df: df["_merge"].isin(["right_only", "both"]),
    )
    .loc[:, ["compound", "definition", "mcmahon", "smalling"]]
    .sort_values("compound")
    .reset_index(drop=True)
)

_factors_dir = mo.notebook_dir() / ".." / "data" / "factors"

if not _factors_dir.exists():
    # Notebook was opened standalone (e.g. `uvx marimo edit --sandbox
    # <gh-pages-url>`), so there's no sibling data/ checkout. Download the
    # same file from the repo into a local cache and use that instead.
    import tempfile as _tempfile
    import urllib.request as _urllib_request
    from pathlib import Path as _Path

    _factors_dir = (
        _Path(_tempfile.gettempdir()) / "egsy-pfas-data" / "factors"
    )
    _factors_dir.mkdir(parents=True, exist_ok=True)
    _dest = _factors_dir / "pfas_tq_benchmarks_epa_aligned.csv"
    if not _dest.exists():
        _urllib_request.urlretrieve(
            "https://raw.githubusercontent.com/egsy-intell/team-project/main",
            _dest,
        )

_tq_benchmark_df = pd.read_csv(
    _factors_dir / "pfas_tq_benchmarks_epa_aligned.csv"
)

all_compound_dict_df = all_compound_dict_df.merge(
    _tq_benchmark_df, on="compound", how="left"
)

```